

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите

Директор Института
кибербезопасности и защиты
информации, д.т.н., проф.

_____ Д. П. Зегжда

«_____» _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

ДИПЛОМНАЯ РАБОТА

РАЗРАБОТКА КРОССПЛАТФОРМЕННОГО ПРИЛОЖЕНИЯ ДЛЯ ОТСЛЕЖИВАНИЯ ДОСТИЖЕНИЙ

по направлению подготовки (специальности)

09.03.01 Информатика и вычислительная техника

Направленность (профиль)

09.03.01_02 Технологии разработки программного обеспечения

Выполнил

студент гр.

А. Е. Плеханов

Руководитель

должность, степень,

звание

Н. В. Богач

Санкт-Петербург – 2026

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Директор ВШ КТиИС

_____ М. А. Полтавцева

«____» _____ 2026 г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Плеханову Артёму Евгеньевичу, группа 5130901/20201

(фамилия, имя, отчество (при наличии), номер группы)

1. Тема работы: Разработка кроссплатформенного мобильного приложения для трекинга достижений с элементами геймификации
2. Срок сдачи студентом законченной работы: 21.05.2026
3. Исходные данные по работе: Требования к функциональности приложения: регистрация и аутентификация пользователя; формирование и редактирование коллекций «паков» достижений; просмотр достижений и шагов выполнения; отметка прогресса с сохранением на сервере; загрузка изображений; локализация интерфейса (русский/английский). Исходные данные к проектированию: спецификация REST API (OpenAPI), описание предметной области «достижения и прогресс», требования к целевым платформам (Android, перспектива Web). Нормативная и учебная литература по многоплатформенной разработке, клиент-серверному взаимодействию и проектированию пользовательских интерфейсов.
4. Содержание работы (перечень подлежащих разработке вопросов):
Анализ предметной области и существующих решений для учёта целей и достижений; постановка задачи и формирование требований к системе.
Выбор технологий: Kotlin Multiplatform, Compose Multiplatform, клиент HTTP, навигация, хранение настроек и токенов, загрузка изображений.

Проектирование архитектуры приложения (слой представления, ViewModel, репозитории, доступ к API).

Реализация модулей: аутентификация, работа с коллекциями паков и достижений, экраны списков и деталей, сценарии создания и редактирования.

Интеграция с сервером: вызовы API, обработка ошибок, синхронизация прогресса.

Реализация пользовательского интерфейса и локализации; обеспечение согласованности состояния при асинхронных операциях.

Тестирование и оценка результатов; формулировка выводов и направлений развития (офлайн-режим, расширение платформ и т.п.).

5. Перечень графического материала (с указанием обязательных чертежей):
Диаграмма компонентов приложения и схема взаимодействия с сервером.
Диаграмма основных экранов (навигация, пользовательские сценарии).
Скриншоты ключевых экранов работающего приложения. Чертежи по ЕСКД для данной темы не требуются.

6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии): Язык программирования Kotlin; Kotlin Multiplatform; Compose Multiplatform; Material Design 3; Gradle (в т.ч. каталог версий зависимостей); Android SDK; среда разработки Android Studio / IntelliJ IDEA; Kotlin Coroutines, Flow, StateFlow; архитектурный подход MVVM, слой репозитория; Navigation 3 (навигация по экранам); Ktor Client (HTTP); REST, JSON; OpenAPI Generator (клиент по спецификации API); OAuth 2.0, Bearer token (использование на клиенте); Coil (изображения); FileKit (выбор файлов); Multiplatform Settings (локальные настройки); локализация интерфейса (ресурсы строк, многоязычность); система контроля версий Git.

7. Консультанты по работе: _____

8. Дата выдачи задания 21.04.2026

Руководитель ВКР

(подпись)

Н. В. Богач

Задание принял к исполнению 21.04.2026

(дата)

Студент

(подпись)

А. Е. Плеханов

РЕФЕРАТ

На №№ с., 1 рисунков, 7 таблиц, №№ приложений.

КЛЮЧЕВЫЕ СЛОВА: КРОССПЛАТФОРМЕННОЕ ПРИЛОЖЕНИЕ, KOTLIN MULTIPLATFORM, COMPOSE MULTIPLATFORM, ГЕЙМИФИКАЦИЯ, ТРЕКИНГ ДОСТИЖЕНИЙ, MVVM, KTOR, OPENAPI GENERATOR.

Тема выпускной квалификационной работы: «Разработка кроссплатформенного мобильного приложения для трекинга достижений с элементами геймификации». Работа посвящена созданию приложения Achi, позволяющего пользователям структурировать цели в виде паков достижений, отслеживать пошаговый прогресс и обмениваться наборами с другими пользователями. Задачи, которые решались в ходе исследования:

1. Провести анализ существующих решений в области трекинга достижений и выявить их функциональные и технические недостатки.
2. Сформулировать функциональные и нефункциональные требования к разрабатываемому приложению.
3. Обосновать выбор технологического стека (Kotlin Multiplatform, Compose Multiplatform, Ktor, Navigation 3, OpenAPI Generator) и спроектировать архитектуру на основе паттерна MVVM с Repository.
4. Реализовать кроссплатформенное приложение с поддержкой иерархической структуры данных (паки → достижения → шаги), аутентификации, синхронизации прогресса с сервером и обмена паками по уникальным кодам.
5. Провести функциональное и кроссплатформенное тестирование разработанного решения.

Разработка выполнена в среде Android Studio с использованием Kotlin Multiplatform и Compose Multiplatform. Сетевой слой сгенерирован из OpenAPI-спецификации, внедрение зависимостей организовано через ручной DI-контейнер AppModule, навигация реализована с помощью Navigation 3. Приложение развёрнуто на платформах Android, iOS и Web (WasmJS).

По результатам работы создано кроссплатформенное приложение, удовлетворяющее сформулированным требованиям: пользователь может управлять коллекцией паков достижений, отслеживать прогресс выполнения шагов с синхронизацией на сервер, создавать собственные паки и добавлять чужие по коду. Полученное решение демонстрирует практическую применимость совре-

менного КМР-стека для разработки продуктов в области персональной продуктивности с элементами геймификации.

ABSTRACT

pages, 1 figures, 7 tables, appendices.

KEYWORDS: CROSS-PLATFORM APPLICATION, KOTLIN MULTIPLATFORM, COMPOSE MULTIPLATFORM, GAMIFICATION, ACHIEVEMENT TRACKING, MVVM, KTOR, OPENAPI GENERATOR.

The subject of the graduate qualification work is «Development of a cross-platform mobile application for achievement tracking with gamification elements». The given work is devoted to the creation of the Achi application, which allows users to structure their goals into achievement packs, track step-by-step progress, and share collections with other users. The research set the following goals:

1. To analyze existing solutions in the field of achievement tracking and identify their functional and technical shortcomings.
2. To formulate functional and non-functional requirements for the application under development.
3. To justify the choice of the technology stack (Kotlin Multiplatform, Compose Multiplatform, Ktor, Navigation 3, OpenAPI Generator) and design the architecture based on the MVVM pattern with a Repository layer.
4. To implement a cross-platform application supporting hierarchical data structure (packs → achievements → steps), authentication, server progress synchronization, and pack sharing via unique codes.
5. To conduct functional and cross-platform testing of the developed solution.

The development was carried out in Android Studio using Kotlin Multiplatform and Compose Multiplatform. The network layer was generated from an OpenAPI specification, dependency injection was organized through a manual DI container `AppModule`, and navigation was implemented using Navigation 3. The application is deployed on Android, iOS, and Web (WasmJS) platforms.

Based on the results of the work, a cross-platform application was created that meets the formulated requirements: the user can manage a collection of achievement packs, track step completion progress with server synchronization, create their own packs, and add others' packs by code. The resulting solution demonstrates the practical applicability of the modern KMP stack for developing products in the field of personal productivity with gamification elements.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
Глава 1. Анализ предметной области и постановка задачи	13
1.1. Обзор предметной области	13
1.2. Анализ существующих решений	14
1.2.1. Habitica	14
1.2.2. Strides	14
1.2.3. Any.do	15
1.2.4. Achievement Box	15
1.2.5. The Achievements	15
1.2.6. Notion и Todoist	16
1.3. Формулировка требований к приложению	17
1.3.1. Пользовательские функции	17
1.3.2. Системные функции	18
1.3.3. Нефункциональные требования	18
1.4. Выводы по главе	18
Глава 2. Выбор технологий и проектирование архитектуры	19
2.1. Выбор среды разработки и языков программирования	19
2.2. Выбор сторонних библиотек и инструментов	20
2.3. Разработка архитектуры приложения	22
2.4. Проектирование структур данных и интеграции с API	24
2.5. Выводы по главе	27
Глава 3. Практическая реализация и тестирование	28
3.1. Практическая реализация приложения	28
3.1.1. Слой данных и генерация API-клиента	28
3.1.2. Слой предметной области (Domain)	29
3.1.3. Слой представления (UI) и ViewModel	30
3.1.4. Внедрение зависимостей и навигация	31
3.2. Тестирование приложения	32
3.2.1. Автоматизированные тесты	32
3.2.2. Функциональное тестирование	32

3.3. Выводы по главе	34
ЗАКЛЮЧЕНИЕ	35
Список использованных источников	37
ПРИЛОЖЕНИЯ	38
Приложение 1. Название приложения	39
Приложение 2. Название приложения	40
Приложение 3. Название приложения	41

ВВЕДЕНИЕ

Актуальность исследования. Современный человек сталкивается с большим количеством личных целей, задач и амбиций. Использование мобильных приложений и цифровых сервисов стало неотъемлемой частью процесса самосовершенствования и организации повседневной деятельности. В условиях растущей информационной нагрузки пользователи испытывают трудности в управлении целями, отслеживании прогресса и поддержании мотивации на протяжении длительного времени.

Геймификация — применение игровых механик в неигровых контекстах — представляет собой активно развивающееся направление в области проектирования пользовательского опыта. Исследования последних лет демонстрируют значительный потенциал геймификации для повышения мотивации и вовлечённости пользователей. Системы достижений, визуализация прогресса и поэтапная декомпозиция целей способствуют формированию устойчивых поведенческих паттернов и поддержанию внутренней мотивации при достижении поставленных задач.

Анализ существующих решений на рынке мобильных приложений для отслеживания достижений и привычек выявил ряд существенных недостатков: нестабильная работа интерфейса, отсутствие возможности обмена пользовательскими наборами достижений, ограниченная кроссплатформенность и зависимость от сетевого подключения. При этом потребность в специализированном инструменте, сочетающем геймифицированный подход к целеполаганию с возможностью совместного использования контента, сохраняет свою актуальность.

Учитывая вышеперечисленное, было принято решение разработать кроссплатформенное приложение — *Achi*, которое позволит пользователям структурировать цели в виде системы достижений, организованных в пакеты, отслеживать прогресс выполнения отдельных шагов и обмениваться наборами достижений с другими пользователями посредством уникальных кодов. Приложение должно обеспечивать синхронизацию данных с сервером, поддерживать различные типы прогресса (бинарный и инкрементальный) и предоставлять единый пользовательский опыт на платформах Android и Web.

Новизна исследования заключается в разработке кроссплатформенного клиентского приложения на базе Kotlin Multiplatform и Compose Multiplatform,

реализующего иерархическую модель достижений (пакеты → достижения → шаги) с механизмом обмена пакетами по коду, оптимистичным обновлением прогресса и автоматической генерацией API-клиента из OpenAPI-спецификации. В отличие от рассмотренных аналогов, предлагаемое решение объединяет стабильный декларативный интерфейс, серверную синхронизацию прогресса и единую кодовую базу для нескольких целевых платформ.

Целью данной работы является разработка кроссплатформенного мобильного приложения для трекинга достижений, применяющего принципы геймификации для повышения мотивации пользователей при достижении персональных целей.

Объектом исследования являются процессы и средства разработки кроссплатформенных мобильных приложений в области персональной продуктивности с элементами геймификации.

Предметом исследования являются методы и инструменты проектирования и реализации кроссплатформенного приложения для отслеживания достижений на базе Kotlin Multiplatform, включая выбор архитектурного паттерна, проектирование структур данных, интеграцию с серверным API и организацию пользовательского интерфейса.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. провести анализ предметной области и существующих решений в области трекинга достижений и привычек, выявить их функциональные и технические недостатки;
2. сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
3. провести анализ технологий кроссплатформенной разработки и обосновать выбор стека (Kotlin Multiplatform, Compose Multiplatform, Ktor, Navigation 3, OpenAPI Generator и др.);
4. спроектировать архитектуру приложения на основе паттерна MVVM с использованием слоя репозитория и ручного внедрения зависимостей;
5. реализовать клиентское приложение с поддержкой аутентификации, управления пакетами достижений, отслеживания прогресса и интеграцией с серверным API;
6. провести функциональное тестирование разработанного решения на целе-

вых платформах.

Гипотеза исследования: использование элементов геймификации (система достижений с поэтапным прогрессом, визуализация выполнения, обмен пакетами между пользователями) в кроссплатформенном приложении, реализованном на Kotlin Multiplatform, позволит создать эффективный инструмент мотивации и целеполагания, устраняющий выявленные недостатки существующих аналогов.

Практическая значимость работы заключается в создании работоспособного кроссплатформенного приложения, которое может быть использовано для организации личных целей и отслеживания прогресса их достижения. Разработанное решение демонстрирует применение современных технологий Kotlin Multiplatform на практике и может служить основой для дальнейшего развития функциональности, включая офлайн-режим.

Публикации: в рамках выполнения работы публикации не осуществлялись.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1. Обзор предметной области

Современный человек сталкивается с большим количеством личных и профессиональных целей. Для поддержания мотивации и систематизации усилий всё шире применяются методы геймификации — использование игровых механик (очки, уровни, награды, прогресс-бары) в неигровых контекстах. Геймификация помогает превращать абстрактные намерения в конкретные, измеримые шаги и повышает вовлечённость пользователя в долгосрочных задачах.

Отдельное направление составляют трекеры привычек и достижений — мобильные и веб-приложения, позволяющие фиксировать выполнение повторяющихся действий или достижение заранее определённых целей. Такие решения варьируются от простых чек-листов до сложных RPG-систем с социальным взаимодействием. Несмотря на разнообразие предложений на рынке, пользователи часто сталкиваются с рядом типичных проблем:

- отсутствие структурированного представления целей в виде коллекций («паков») с пошаговым прогрессом;
- невозможность делиться готовыми наборами достижений с другими пользователями;
- перегруженный игровой интерфейс либо, напротив, недостаток специализированных функций;
- зависимость от одной платформы или нестабильная работа кроссплатформенных решений.

Мобильные устройства и современные кроссплатформенные технологии разработки позволяют создавать специализированные приложения, объединяющие учёт достижений, визуализацию прогресса и синхронизацию данных между устройствами. Разрабатываемое приложение **Achi** направлено на решение указанных задач: пользователь может собирать коллекции достижений, отслеживать выполнение шагов, создавать собственные «паки» и добавлять в коллекцию наборы, созданные другими участниками.

1.2. Анализ существующих решений

В данном разделе представлен анализ современных решений для мониторинга привычек, задач и достижений. Рассмотрены как специализированные приложения, так и универсальные инструменты продуктивности. Особое внимание уделяется функциональным возможностям, качеству пользовательского интерфейса и пригодности для сценария «структурированные коллекции достижений с пошаговым прогрессом».

1.2.1. Habitica

Habitica — популярное приложение для формирования привычек и отслеживания задач, использующее элементы ролевых игр (RPG) [3]. Пользователь выполняет привычки, ежедневные задачи и пункты списка дел, получая за это опыт, золото и экипировку для игрового персонажа. Предусмотрено объединение в группы для совместного выполнения квестов.

Преимущества: высокий уровень вовлечённости благодаря геймификации; развитое сообщество; бесплатный базовый функционал.

Недостатки: перегруженный игровой интерфейс, отвлекающий от реальных задач; отсутствие концепции «паков» достижений с пошаговым прогрессом; сложность адаптации для серьёзных или профессиональных целей.

1.2.2. Strides

Strides — трекер привычек и целей с акцентом на аналитику и визуализацию прогресса [5]. Поддерживаются четыре типа целей: привычка, цель, среднее значение и проект. Приложение предоставляет подробную статистику, графики и напоминания.

Преимущества: качественная визуализация данных; гибкая настройка целей; стабильный интерфейс.

Недостатки: ограниченный бесплатный функционал (лимит на количество целей); отсутствие социальной составляющей и возможности делиться наборами целей; преимущественная ориентация на платформу iOS.

1.2.3. Any.do

Any.do — менеджер задач и календарь с поддержкой списков, напоминаний и голосового ввода [2]. Приложение ориентировано на повседневное планирование и управление делами.

Преимущества: простой и понятный интерфейс; кроссплатформенность; интеграция с календарём.

Недостатки: отсутствие специализации на достижениях; нет концепции коллекций или пошагового прогресса внутри «награды»; ограниченные возможности совместного использования пользовательских наборов задач.

1.2.4. Achievement Box

Achievement Box — приложение с геймифицированным подходом к формированию полезных привычек [1]. За выполнение «хороших» привычек пользователь зарабатывает монеты, которые можно тратить в магазине наград. Предусмотрена статистика, графики и резервное копирование через сервисы Google.

Преимущества: стабильная работа и корректное отображение интерфейса; открытый исходный код; развитая система личной статистики.

Недостатки: достижения смешаны с понятием «привычек»; отсутствует возможность передавать созданные наборы другим пользователям; нет профиля и социальных функций; ограниченный выбор иконок (нельзя загрузить собственные изображения).

1.2.5. The Achievements

The Achievements — приложение для создания, отправки и получения достижений с социальными функциями [6]. Поддерживается генерация достижений с помощью ИИ, отправка через QR-коды и ссылки, а также лайки, комментарии и подписки.

Преимущества: развитая система пользователей и профилей; возможность выдавать достижения другим; кастомизация стилей.

Недостатки: проблемы с интерфейсом (некорректные отступы, артефакты вёрстки); полная зависимость от интернета; нельзя «взять» достижение для самостоятельного выполнения — решение принимает только создатель; отсут-

ствуется пошаговый прогресс внутри достижения.

1.2.6. Notion и Todoist

Notion — универсальный инструмент для управления задачами, заметками и проектами [4]. Пользователь может организовать информацию в виде страниц, списков и таблиц, настроив собственную систему учёта прогресса. Todoist — менеджер задач с поддержкой проектов, меток и фильтров [7], ориентированный на учёт дедлайнов и повседневных дел.

Преимущества Notion: высокая степень персонализации; подходит для сложных проектов; совместная работа.

Недостатки Notion: сложность освоения; отсутствие встроенной геймификации и специализированного интерфейса «достижений»; требуется ручная настройка всех таблиц и связей.

Преимущества Todoist: удобное управление задачами; интеграция с календарями; кроссплатформенность.

Недостатки Todoist: не специализируется на «достижениях» или «коллекциях»; отсутствует визуальное представление прогресса в виде наград или бейджей с пошаговой структурой.

Результаты сравнения аналогов представлены в табл. 1.1.

Таблица 1.1

Сравнительный анализ аналогов

Критерий	Habitica	Strides	Any.do	Achiev. Box	The Achiev.	Notion	Todoist	Achi
Создание своих достижений	Нет	Нет	Нет	Частично	Да	Вручную	Нет	Да
Шеринг наборов (паков)	Нет	Нет	Нет	Нет	Да	Шаблоны	Нет	Да
Пошаговый прогресс	Нет	Частично	Нет	Нет	Нет	Вручную	Нет	Да
Профиль и социальность	Да	Нет	Нет	Нет	Да	Да	Да	Да
Стабильность UI/UX	Средняя	Высокая	Высокая	Высокая	Низкая	Средняя	Высокая	—
Кроссплатформенность	Да	Частично	Да	Android	Да	Да	Да	Да
Геймификация	Да	Нет	Нет	Да	Нет	Нет	Частично	Частично

Таким образом, существующие аналоги обладают разными преимуществами и недостатками. Большинство из них либо не предоставляют возможности создавать структурированные коллекции («паки») достижений с пошаговым прогрессом, либо не поддерживают их передачу другим пользователям. Популярные трекеры привычек (Habitica, Strides) и task-менеджеры (Todoist, Any.do) не сфокусированы на концепции «достижений» как самостоятельной сущности. Это подтверждает актуальность разработки специализированного кроссплатформенного приложения Achi.

1.3. Формулировка требований к приложению

На основе анализа предметной области и существующих решений сформулированы требования к разрабатываемому приложению Achi.

1.3.1. Пользовательские функции

Управление учётной записью:

- регистрация и авторизация пользователя для синхронизации прогресса и коллекций паков с сервером;
- сохранение сессии (токен авторизации) и возможность выхода из аккаунта.

Работа с достижениями и паками:

- просмотр списка паков в коллекции пользователя;
- детализация достижений внутри пака: описание, изображения, текущий прогресс;
- отслеживание прогресса: отметка выполнения отдельных шагов (подзадач) с автоматическим расчётом общего прогресса и синхронизацией с сервером;
- добавление паков по уникальному коду, созданных другими пользователями;
- создание собственных паков: добавление достижений, настройка шагов, загрузка изображений-превью;
- редактирование, копирование и удаление созданных паков.

Интерфейс и навигация:

- нижняя панель навигации (Bottom Nav) с разделами «Добавление», «Мои достижения», «Профиль»;
- локализация интерфейса (русский и английский языки);
- просмотр пользовательского соглашения и политики конфиденциальности.

1.3.2. Системные функции

- синхронизация данных: фоновая отправка прогресса шагов на сервер с оптимистичным обновлением пользовательского интерфейса;
- взаимодействие с REST API через сгенерированные клиенты (OpenAPI);
- отладочная панель для переключения хоста API и быстрой авторизации (только в отладочных сборках).

1.3.3. Нефункциональные требования

Архитектура и производительность:

- построение приложения по паттерну MVVM с выделением слоя Repository;
- оптимизированная загрузка и кэширование изображений.

Кроссплатформенность:

- единая кодовая база для бизнес-логики и пользовательского интерфейса;
- поддержка платформ: Android (основная), Web/WasmJS (вторичная), iOS.

Пользовательский интерфейс:

- использование дизайн-системы Material 3;
- адаптивность под различные размеры экранов;
- минимальная версия Android — API 24 (Android 7.0).

1.4. Выводы по главе

1. Рассмотрена предметная область геймификации и трекинга достижений. Выявлена потребность в специализированном решении, объединяющем структурированные коллекции достижений, пошаговый прогресс и возможность обмена наборами между пользователями.
2. Проведён сравнительный анализ восьми аналогов (Habitica, Strides, Any.do, Achievement Box, The Achievements, Notion, Todoist). Ни одно из рассмотренных решений не сочетает полностью функции создания «паков» с пошаговым прогрессом, шеринга и кроссплатформенной реализации.
3. На основе анализа сформулированы функциональные и нефункциональные требования к приложению Achi, определяющие ключевые сценарии: регистрация, управление коллекциями, отслеживание прогресса с серверной синхронизацией и создание собственных паков.

ГЛАВА 2. ВЫБОР ТЕХНОЛОГИЙ И ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ

2.1. Выбор среды разработки и языков программирования

Разработка кроссплатформенного мобильного приложения предполагает выбор технологии, позволяющей совместить единую бизнес-логику с нативной производительностью на целевых платформах. Для решения данной задачи рассматривались три основных подхода: отдельная нативная разработка (Kotlin для Android, Swift для iOS), гибридные фреймворки (React Native, Flutter) и технология Kotlin Multiplatform (KMP).

При отдельной нативной разработке каждая платформа реализуется независимо. Такой подход обеспечивает максимальную гибкость и полный доступ к платформенным API, однако требует дублирования логики, что увеличивает трудозатраты на сопровождение и повышает риск расхождения функциональности между версиями приложения.

Гибридные фреймворки позволяют использовать единую кодовую базу, но зачастую полагаются на собственный рендеринг интерфейса или на JavaScript-мост, что может ограничивать интеграцию с нативными компонентами и влиять на производительность.

Kotlin Multiplatform предлагает иной компромисс: общий код на языке Kotlin компилируется в нативные бинарные файлы для каждой целевой платформы, а платформенно-специфичные части выделяются через механизм `expect/actual`. Данный подход позволяет переиспользовать до 90% кодовой базы при сохранении доступа к нативным API там, где это необходимо.

Сравнительная характеристика рассмотренных подходов представлена в табл. 2.1.

Таблица 2.1

Сравнение подходов к кроссплатформенной разработке мобильных приложений

Критерий	Нативная разработка	React Native / Flutter	Kotlin Multiplatform
Переиспользование кода	Низкое	Высокое	Высокое
Производительность	Максимальная	Средняя	Высокая (нативная компиляция)
Доступ к платформенным API	Полный	Ограниченный	Полный (через <code>actual</code>)

Продолжение табл. 2.1

Критерий	Нативная разработка	React Native / Flutter	Kotlin Multiplatform
Единый язык разработки	Нет	Частично (JS/Dart + нативный код)	Да (Kotlin)
Зрелость экосистемы	Высокая	Высокая	Растущая

Для проекта Achi выбран Kotlin Multiplatform в связи с необходимостью поддержки платформ Android, Web (WasmJS) и iOS из единого модуля `shared` без переписывания бизнес-логики. Язык Kotlin обеспечивает статическую типизацию, поддержку корутин для асинхронных операций и тесную интеграцию с экосистемой JetBrains.

В качестве среды разработки используется Android Studio с плагинами Kotlin Multiplatform и Compose Multiplatform. Сборка проекта осуществляется системой Gradle; версии ключевых компонентов зафиксированы в каталоге зависимостей (`libs.versions.toml`): Kotlin 2.3.0, Compose Multiplatform 1.10.0, Ktor 3.3.3.

Целевые платформы приложения Achi и их текущий статус приведены в табл. 2.2.

Таблица 2.2

Целевые платформы приложения Achi

Платформа	Статус	Минимальная версия
Android	Активно (Stable)	Android 7.0 (API 24)
Web (WasmJS)	Активно (Beta)	Браузеры с поддержкой WebAssembly
iOS	Активно (Stable)	iOS 14+

2.2. Выбор сторонних библиотек и инструментов

Помимо базовой платформы KMP, для реализации приложения Achi используется набор сторонних библиотек, отобранных с учётом совместимости с Kotlin Multiplatform, зрелости и соответствия функциональным требованиям. Обоснование выбора ключевых компонентов приведено в табл. 2.3.

Таблица 2.3

Технологический стек приложения Achi

Компонент	Технология	Обоснование выбора
UI-фреймворк	Compose Multiplatform	Декларативный подход; единый UI-код; Material 3
HTTP-клиент	Ktor Client	Нативная поддержка КМР; корутины; модульность
Навигация	Navigation 3 (navigation3-ui)	Типобезопасная КМР-навигация; сериализация маршрутов
Генерация API	OpenAPI Generator	Автоматическая генерация клиента из спецификации
Загрузка изображений	Coil 3	КМР-совместимость; интеграция с Ktor
Локальное хранилище	Multiplatform Settings	Сохранение токена авторизации на всех платформах
Выбор файлов	FileKit	КМР-диалоги выбора изображений при создании паков
Сериализация	kotlinx.serialization	Интеграция с Ktor и Navigation 3

Compose Multiplatform — декларативный UI-фреймворк, основанный на парадигме реактивного обновления интерфейса при изменении состояния. В отличие от XML-вёрстки Android или SwiftUI для iOS, Compose Multiplatform позволяет описывать пользовательский интерфейс на языке Kotlin и переиспользовать экраны на всех целевых платформах. Для приложения Achi используется дизайн-система Material 3 с поддержкой динамических цветов Material You на Android.

Ktor Client — асинхронный HTTP-клиент, разработанный JetBrains с первоначальной ориентацией на Kotlin. Ktor поддерживает модульную архитектуру (Content Negotiation, Logging) и предоставляет платформенные движки: OkHttp для Android, CIO для WasmJS. В проекте Achi Ktor используется как транспортный слой для сгенерированных OpenAPI-клиентов.

Navigation 3 (`org.jetbrains.androidx.navigation3:navigation3`) — новая библиотека навигации для Compose Multiplatform, заменяющая Jetpack Navigation Compose в КМР-проектах. Маршруты реализуют интерфейс `NavKey` и сериализуются через `kotlinx.serialization`, что обеспечивает типобезопасность и сохранение стека навигации. Для Achi определены маршруты списка паков, списка достижений, деталей достижения, создания

пака, профиля и настроек.

OpenAPI Generator (версия 7.18.0) генерирует Ktor-клиент и модели данных из файла спецификации `Achi_openapi.json`. Автоматическая генерация снижает вероятность ошибок при интеграции с REST API и упрощает синхронизацию клиента при изменении серверного контракта. Сгенерированные схемы (`AchievementSchema`, `AchievementPackSchema`) преобразуются в доменные модели через функции-мапперы.

Coil 3 обеспечивает асинхронную загрузку и кэширование изображений (превью паков и достижений) с использованием Ktor Network Fetcher. **Multiplatform Settings** применяется для персистентного хранения OAuth2-токена и данных сессии пользователя. **FileKit** предоставляет кроссплатформенные диалоги выбора файлов при загрузке изображений в процессе создания паков достижений.

Следует отметить ряд ограничений выбранного стека: Compose Multiplatform и Kotlin/Wasm для Web-платформы находятся в стадии активного развития; не все популярные Android- и iOS-библиотеки имеют KMP-аналоги; размер итогового бинарного файла KMP-приложения превышает размер чисто нативного. Указанные факторы учтены при планировании архитектуры: приоритет отдан мобильным платформам (Android и iOS), а Web-версия рассматривается как дополнительный канал распространения.

2.3. Разработка архитектуры приложения

Архитектура приложения Achi построена на паттерне MVVM (Model–View–ViewModel) с дополнительным слоем Repository и ручным контейнером внедрения зависимостей. Данное решение обеспечивает разделение ответственности между слоями, тестируемость компонентов и совместимость с реактивной моделью Compose.

Структура слоёв приложения представлена на рис. 2.1.

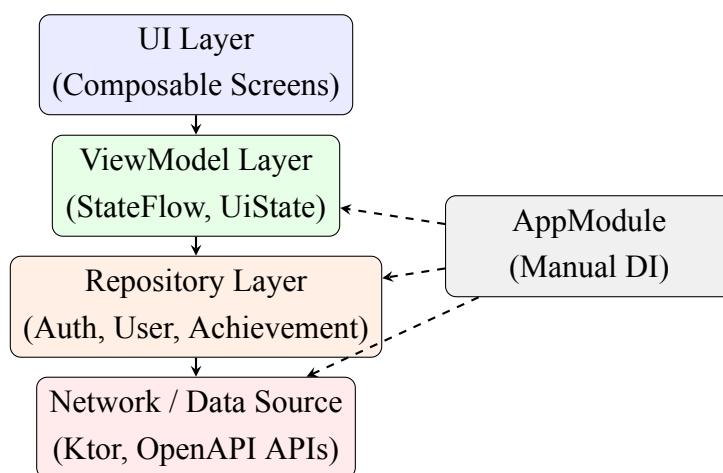


Рис. 2.1. Слоистая архитектура приложения Achi

Слой представления (View) содержит Composable-функции экранов, расположенные в пакете `ui/`. Экраны являются декларативными и не содержат бизнес-логики: они наблюдают состояние `ViewModel` через `collectAsState()` и делегируют обработку пользовательских действий методам `ViewModel`. Навигация между экранами реализована через `NavDisplay` и `NavBackStack` библиотеки `Navigation 3`.

Слой ViewModel располагается рядом с соответствующими экранами и наследует `androidx.lifecycle.ViewModel`. `ViewModel` хранит состояние интерфейса в `MutableStateFlow` и предоставляет неизменяемый `StateFlow<UiState>` для наблюдения из UI. Для одноразовых событий (сообщения `Snackbar`, навигация) используются `Channel` и `Flow`. Асинхронные операции выполняются в `viewModelScope` с использованием корутин Kotlin.

Слой Repository инкапсулирует доступ к данным и скрывает детали сетевого взаимодействия от `ViewModel`. В проекте определены четыре репозитория:

- `AuthRepository` — аутентификация (логин, регистрация, выход), управление OAuth2-токеном;
- `UserRepository` — коллекция паков пользователя и синхронизация прогресса;
- `AchievementRepository` — операции CRUD над достижениями;
- `AchievementPackRepository` — операции CRUD над паками, загрузка изображений.

Репозитории реализуют паттерн `Repository`: интерфейс определяет контракт, а реализация (`*Impl`) обращается к сгенерированным API-клиентам.

Преобразование сетевых моделей в доменные выполняется функциями-мапперами в пакете `data/network/`.

Внедрение зависимостей реализовано вручную через класс `AppModule`, без использования фреймворков `Koin` или `Dagger`. Контейнер создаёт API-клиенты (`AchievementsApi`, `PacksApi`, `AuthenticationApi` и др.) и репозитории (ленивая инициализация через `by lazy`). Экземпляр `AppModule` создаётся в корневом `Composable AchAppNav` и передаётся в провайдер навигации для создания `ViewModel`.

Выбор `Manual DI` обусловлен умеренным масштабом проекта: граф зависимостей содержит около десяти компонентов, что делает явное управление зависимостями прозрачным и не требующим дополнительной конфигурации. При росте сложности проекта возможен переход на `DI-фреймворк`.

Ключевые архитектурные решения приложения `Achi`:

1. единый модуль `shared` для всего общего кода;
2. оптимистичные обновления прогресса (UI обновляется немедленно, синхронизация с сервером — в фоне);
3. проверка состояния авторизации перед операциями, требующими учётной записи;
4. локализация через `composeResources` (английский и русский языки) с абстракцией `UiText` для сообщений из `ViewModel`.

2.4. Проектирование структур данных и интеграции с API

Предметная область приложения `Achi` описывается иерархией сущностей: *пак достижений* (`AchievementPack`) содержит список *достижений* (`Achievement`), каждое из которых может быть декомпозировано на *шаги* (`AchievementStep`) с отслеживаемым *прогрессом* (`StepProgress`).

Доменные модели определены в пакете `data/model/` и не зависят от сетевого слоя. Серверные модели (`*Schema`) генерируются `OpenAPI Generator` и преобразуются в доменные через функции расширения (`toAchievement()`, `toAchievementPack()` и др.).

Модель пака достижений представлена классом `AchievementPack`:

Листинг 2.1. Доменная модель пака достижений

```
1 data class AchievementPack(
```



```

2     val id: String,
3     val name: String,
4     val count: Int,
5     val achievementIds: List<String>,
6     val previewImageUrl: String? = null,
7     val code: String,
8     val creatorId: String = ""
9 )

```

Поле `code` — уникальный код для добавления пака в коллекцию другого пользователя. Поле `achievementIds` содержит идентификаторы входящих достижений; полный список объектов `Achievement` загружается отдельными запросами.

Модель достижения и связанных сущностей:

Листинг 2.2. Доменные модели достижения, шага и прогресса

```

1 data class Achievement(
2     val id: String,
3     val title: String,
4     val shortDescription: String,
5     val longDescription: String? = null,
6     val steps: List<AchievementStep>,
7     val previewImageUrl: String? = null,
8     val imageUrl: String? = null,
9     val isCompleted: Boolean = false,
10    val creatorId: String = ""
11 )
12
13 data class AchievementStep(
14     val id: String,
15     val description: String,
16     val progress: StepProgress = StepProgress(),
17 )
18
19 data class StepProgress(

```

```

20     val substepsDone: Int = 0,
21     val substepsAmount: Int = 1
22 )

```

Модель `StepProgress` поддерживает инкрементальный прогресс: пользователь может выполнить N из M подшагов (например, «прочитать 3 из 5 глав»). Для достижений без шагов (`steps.isEmpty()`) используется бинарный флаг `isCompleted`. Вычисляемые свойства `progress`, `isDone` и `stepsDone` определяют общий прогресс достижения на основе состояния шагов.

Связь доменных моделей с REST API реализована через набор сгенерированных клиентов, перечисленных в табл. 2.4.

Таблица 2.4

API-клиенты приложения Achi и их назначение

API-клиент	Основные операции	Доменные сущности
<code>AuthenticationApi</code>	Получение OAuth2-токена	—
<code>UsersApi</code>	Регистрация пользователя	—
<code>PacksApi</code>	CRUD паков, поиск по коду	<code>AchievementPack</code>
<code>AchievementsApi</code>	CRUD достижений	<code>Achievement</code>
<code>UserCollectionApi</code>	Коллекция паков пользователя	<code>AchievementPack</code>
<code>UserProgressApi</code>	Синхронизация прогресса шагов	<code>StepProgress</code>
<code>UploadApi</code>	Загрузка изображений	—

Базовый URL сервера задаётся в объекте `ApiConfig` и может изменяться в режиме отладки через панель `Debug Panel`. Токен авторизации инжектируется во все API-клиенты через `AuthRepository` при успешном входе и удаляется при выходе.

Состояние аутентификации описывается `data class AuthState`, содержащим флаги `isLoggedIn`, `username`, `userId`, `accessToken` и поля для отображения ошибок. Репозиторий предоставляет `StateFlow<AuthState>`, на который подписываются `ViewModel` для адаптации интерфейса (отображение формы входа или данных профиля).

Поток данных при обновлении прогресса шага иллюстрирует интеграцию слоёв:

1. пользователь нажимает кнопку увеличения прогресса на экране деталей до-

- стижения;
- 2. `AchievementDetailsViewModel` оптимистично обновляет локальное состояние `UiState`;
- 3. при наличии активной сессии `ViewModel` вызывает `UserRepository.updateStepProgress()`;
- 4. репозиторий обращается к `UserProgressApi` и сохраняет результат;
- 5. при ошибке сетевого запроса пользователю отображается сообщение через `Snackbar`.

Таким образом, доменные модели отделены от транспортного слоя, а `Repository` обеспечивает единую точку доступа к данным для всех `ViewModel` приложения.

2.5. Выводы по главе

- 1. Для разработки кроссплатформенного приложения `Achi` выбран `Kotlin Multiplatform` как технология, обеспечивающая переиспользование кода между `Android`, `iOS` и `Web` при сохранении нативной производительности.
- 2. Сформирован технологический стек на базе `Compose Multiplatform`, `Ktor`, `Navigation 3`, `OpenAPI Generator` и вспомогательных КМР-библиотек (`Coil`, `Multiplatform Settings`, `FileKit`), соответствующий функциональным и нефункциональным требованиям.
- 3. Спроектирована слоистая архитектура `MVVM` с `Repository pattern` и ручным внедрением зависимостей через `AppModule`, обеспечивающая разделение ответственности и тестируемость компонентов.
- 4. Определены доменные модели `AchievementPack`, `Achievement`, `AchievementStep` и `StepProgress`, а также схема интеграции с `REST API` через сгенерированные `Ktor`-клиенты и функции-мапперы.

ГЛАВА 3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1. Практическая реализация приложения

В данной главе описывается практическая реализация кроссплатформенного приложения *Achi* — клиента для учёта достижений и синхронизации прогресса с сервером. Исходный код размещён в модуле `shared` и организован по принципу многослойной архитектуры MVVM с паттерном Repository, спроектированной во второй главе. Общий код для Android, iOS и Web (WasmJS) находится в `commonMain`; платформенные особенности вынесены в `androidMain`, `iosMain` и `wasmJsMain`.

3.1.1. Слой данных и генерация API-клиента

Слой данных отвечает за взаимодействие с REST API сервера, преобразование сетевых моделей в доменные и кэширование результатов в репозиториях.

Генерация API. Спецификация `OpenAPI` (`Achi_openapi.json`) используется для автоматической генерации типизированных Ktor-клиентов с помощью `OpenAPI Generator 7.18.0`. Задача `openApiGenerate` в `build.gradle.kts` настроена на генерацию Kotlin Multiplatform-кода в каталог `build/generated/openapi` с пакетом `com.plezha.achi.shared.data.network`. Сгенерированные классы включают API-интерфейсы (`AchievementsApi`, `PacksApi`, `AuthenticationApi`, `UsersApi`, `UploadApi`, `UserCollectionApi`, `UserProgressApi`) и схемы данных (`AchievementSchema`, `AchievementPackSchema` и др.). Компиляция модуля зависит от задачи генерации, что гарантирует актуальность клиента при изменении спецификации.

Маппинг сетевых моделей. Файл `Mappers.kt` содержит функции расширения, преобразующие схемы API в доменные типы. При маппинге относительные URL изображений дополняются базовым адресом сервера из `ApiConfig.baseUrl`. Пример преобразования достижения:

Листинг 3.3. Маппинг сетевой схемы в доменную модель

```
1 fun AchievementSchema.toAchievement() = Achievement(
2     id = id,
3     title = title,
```

```

4     shortDescription = shortDescription,
5     longDescription = longDescription,
6     steps = steps.map { it.toAchievementStep() },
7     previewImageUrl = resolveImageUrl(previewImageUrl),
8     imageUrl = resolveImageUrl(imageUrl),
9     creatorId = creatorId
10 )

```

Репозитории. Бизнес-операции инкапсулируются в классах репозитория:

- `AuthRepository` — аутентификация (логин, регистрация, выход), восстановление сессии из локального хранилища, установка bearer-токена на все API-клиенты;
- `AchievementRepository` / `AchievementPackRepository` — CRUD-операции над достижениями и пакетами, загрузка изображений;
- `UserRepository` — коллекция пакетов пользователя и синхронизация прогресса по шагам с сервером.

Репозитории скрывают детали HTTP-запросов и обработку ответов (расширение `check()` для проверки кодов ответа). `AuthRepository` сохраняет токен, имя пользователя и идентификатор в `multiplatform-settings`, что обеспечивает сохранение сессии между запусками приложения. Состояние авторизации публикуется через `StateFlow<AuthState>`.

3.1.2. Слой предметной области (Domain)

Доменные модели расположены в пакете `data.model` и не зависят от UI и сетевого слоя. Основные типы приведены в табл. 3.1.

Таблица 3.1

Доменные модели приложения Achi

Модель	Ключевые поля	Вычисляемые свойства
<code>AchievementPack</code>	<code>id</code> , <code>name</code> , <code>count</code> , <code>achievementIds</code> , <code>code</code> , <code>previewImageUrl</code>	—
<code>Achievement</code>	<code>id</code> , <code>title</code> , <code>steps</code> , <code>isCompleted</code> , <code>imageUrl</code>	<code>progress</code> , <code>isDone</code>
<code>AchievementStep</code>	<code>id</code> , <code>description</code> , <code>progress</code>	<code>isDone</code>
<code>StepProgress</code>	<code>substepsDone</code> , <code>substepsAmount</code>	<code>progressFloat</code> , <code>isCompleted</code>

Модель `Achievement` поддерживает два режима завершения: по шагам (прогресс вычисляется как среднее по шагам) и без шагов (флаг `isCompleted`). Валидация прогресса выполняется в `init`-блоке `StepProgress`: значение `substepsDone` ограничено диапазоном `[0; substepsAmount]`. Вспомогательные функции (`overallProgress()`, `completedCount()`) позволяют агрегировать статистику по списку достижений на уровне пакета.

3.1.3. Слой представления (UI) и `ViewModel`

Пользовательский интерфейс реализован на `Compose Multiplatform` с `Material 3`. Экраны сгруппированы по функциональным областям: `ui/list` (списки пакетов и достижений, детали), `ui/add` (создание и редактирование пакетов), `ui/profile`, `ui/settings`, `ui/debug`.

Паттерн `ViewModel`. Каждый экран с бизнес-логикой сопровождается `ViewModel`, получающей зависимости через конструктор. Состояние UI хранится в `MutableStateFlow` и экспонируется как неизменяемый `StateFlow`. Одноразовые события (навигация, сообщения) передаются через `Channel` и преобразуются в `Flow`.

Локализация. Для поддержки английского и русского языков `ViewModel` формируют сообщения через абстракцию `UiText` (сырая строка или ресурс `StringResource`), а `Composable`-функции разрешают их в локализованный текст через `asString()` или `resolve()`.

Оптимистичные обновления. При изменении прогресса шага или переключении завершения достижения UI обновляется немедленно; синхронизация с сервером выполняется асинхронно в `viewModelScope`. При ошибке сети пользователю показывается сообщение через `Snackbar`, локальное состояние при этом сохраняется до следующей успешной синхронизации. Пример из `AchievementDetailsViewModel`:

Листинг 3.4. Оптимистичное обновление прогресса шага

```

1 // Update UI immediately (optimistic update)
2 _uiState.update {
3     val achievement = it.achievement!!
4     it.copy(achievement = localUpdate(achievement, stepId))
5 }

```

```

6 // Sync to server if logged in
7 if (isLoggedIn) {
8     viewModelScope.launch {
9         userRepository.updateStepProgress(
10             achievementId, stepId, newSubstepsDone)
11     }
12 }

```

Загрузка изображений. В точке входа `AchiApp` настраивается глобальный `ImageLoader` библиотеки `Coil 3` с `Ktor Network Fetcher` и поддержкой локальных файлов через `FileKit`, что обеспечивает единообразную работу с првью и загруженными изображениями на всех платформах.

3.1.4. Внедрение зависимостей и навигация

Manual DI через AppModule. Вместо фреймворков `Koin` или `Dagger` в проекте используется ручной контейнер зависимостей `AppModule`. Он создаётся в корневом `Composable` `AchiAppNav` с привязкой к текущему `baseUrl` (для переключения сервера в `debug`-сборке). Контейнер содержит:

- семь API-клиентов, инициализируемых с `HttpClientEngine` платформы (`expect/actual`);
- четыре репозитория, создаваемых лениво (`by lazy`) для отложенной инициализации.

`ViewModel` создаются в `App.kt` или в `NavGraph.kt` через `remember` с передачей репозитория из `AppModule`. Такой подход минимизирует количество зависимостей, сохраняет прозрачность графа объектов и не требует рефлексии или кодогенерации.

Navigation 3. Для навигации применяется библиотека `navigation3-ui` (версия `1.0.0-alpha05`), а не классический `Jetpack Navigation Compose`. Маршруты определены как типизированные `NavKey` с аннотацией `@Serializable` в файле `Routes.kt`. Полиморфная сериализация настроена в `navSavedStateConfig` для сохранения стека при смене конфигурации.

Стек навигации управляется через `rememberNavController`; отображение экранов — через `NavDisplay` и `entryProvider`. Функция `createNavController` регистрирует все экраны приложения и связывает их с `ViewModel`. Нижняя панель (`BottomNavigationBar`) содержит три

вкладки: «Добавить», «Достижения» (стартовый экран), «Профиль». Вложенные маршруты (список достижений, детали, создание пакета, настройки, debug-панель) добавляются в стек поверх соответствующей вкладки.

Схема потока данных и навигации:

1. Composable считывает `uiState` из `ViewModel` через `collectAsState`;
2. действие пользователя вызывает метод `ViewModel`;
3. `ViewModel` обращается к репозиторию, обновляет `StateFlow`;
4. при необходимости `ViewModel` отправляет событие навигации в `Channel`;
5. `LaunchedEffect` в `NavGraph` обрабатывает событие и изменяет `NavBackStack`.

3.2. Тестирование приложения

Тестирование проводилось на двух уровнях: автоматизированные модульные тесты и ручное функциональное тестирование на целевых платформах.

3.2.1. Автоматизированные тесты

В модуле `shared` подготовлена инфраструктура для unit-тестов (`androidHostTest`) и инструментальных тестов `Android` (`androidDeviceTest`). На момент написания работы размещены базовые примеры (`ExampleUnitTest`, `ExampleInstrumentedTest`), подтверждающие корректность конфигурации тестовых таргетов. Дальнейшее расширение покрытия предполагает тестирование мапперов, логики `StepProgress` и `ViewModel` с использованием моков репозиториев.

3.2.2. Функциональное тестирование

Функциональное тестирование выполнялось вручную по сценариям, охватывающим основные пользовательские потоки. Результаты сведены в табл. 3.2.

Таблица 3.2

Результаты функционального тестирования

№ п/п	Сценарий	Ожидаемый результат	Платформы
1	Регистрация и вход	Сессия сохраняется, отображается имя пользователя	Android, Web, iOS
2	Просмотр коллекции пакетов	Загружается список пакетов авторизованного пользователя	Android, Web, iOS
3	Добавление пакета по коду	Пакет появляется в коллекции	Android, Web, iOS
4	Отметка прогресса шага	UI обновляется мгновенно, данные синхронизируются с сервером	Android, Web, iOS
5	Создание пакета с изображением	Пакет создаётся, превью отображается корректно	Android, Web, iOS
6	Редактирование и удаление пакета	Изменения сохраняются на сервере, список обновляется	Android, Web, iOS
7	Переключение языка системы	Интерфейс отображается на EN или RU	Android, Web, iOS
8	Debug-панель (debug-сборка)	Переключение хоста API, быстрый вход	Android

Кроссплатформенная проверка. Общий UI-код в `commonMain` обеспечивает идентичное поведение на Android, Web и iOS. Отличия ограничены платформенными адаптерами: `HttpClientEngine`, определение флага `isDebug`, поддержка Material You на Android. Сборка Web-версии выполнялась командой `./gradlew wasmJsBrowserDevelopmentRun`; Android — `./gradlew installDebug`; iOS — через Xcode. Критические сценарии (авторизация, синхронизация прогресса, навигация между экранами) прошли успешно на всех платформах.

Нефункциональные аспекты. Проверялась корректность работы при отсутствии сети (локальное отображение последнего состояния, сообщения об ошибках синхронизации), сохранение сессии после перезапуска приложения и корректность отображения при светлой и тёмной темах Material 3.

3.3. Выводы по главе

1. Реализован слой данных с автогенерацией API-клиента из OpenAPI-спецификации, маппингом схем в доменные модели и репозиториями для инкапсуляции сетевого взаимодействия.
2. Доменный слой содержит типизированные модели достижений, пакетов и прогресса с валидацией и вычисляемыми свойствами, не зависящими от UI и транспорта.
3. Слой представления построен на Compose Multiplatform с ViewModel, StateFlow, локализацией через UiText и оптимистичными обновлениями прогресса.
4. Внедрение зависимостей выполнено вручную через AppModule; навигация реализована на Navigation 3 с типизированными маршрутами и единым NavGraph.
5. Функциональное тестирование подтвердило работоспособность основных сценариев на Android, Web и iOS; заложена основа для расширения автоматизированного покрытия.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы был проведён комплексный анализ предметной области трекинга достижений и геймификации персональной продуктивности. Исследование существующих решений (Habitica, Strides, Any.do, а также специализированных приложений из Google Play) позволило выявить их ключевые преимущества и недостатки: отсутствие стабильного кроссплатформенного продукта с возможностью создания и обмена пользовательскими наборами достижений, ограниченная поддержка пошагового прогресса и зависимость ряда решений от постоянного сетевого подключения. Полученные результаты легли в основу формулировки функциональных и нефункциональных требований к разрабатываемому приложению Achi.

На следующем этапе были проанализированы технологии, позволяющие реализовать приложение в соответствии с сформулированными требованиями. В качестве основы выбран стек Kotlin Multiplatform и Compose Multiplatform, обеспечивающий единую кодовую базу для мобильных платформ (Android, iOS) и Web (WasmJS). Для сетевого взаимодействия применён Ktor Client, для загрузки изображений — Coil 3, для навигации — библиотека Navigation 3 с типобезопасными маршрутами (NavKey), для генерации API-клиента — OpenAPI Generator на основе спецификации `Achi_openapi.json`. Архитектура приложения построена по паттерну MVVM с выделением слоя репозитория; внедрение зависимостей реализовано вручную через контейнер AppModule, что обеспечивает прозрачность связей между компонентами без использования сторонних DI-фреймворков.

В рамках практической реализации разработаны доменные модели данных (пакеты достижений, достижения, шаги с инкрементальным и бинарным прогрессом), репозитории для работы с API и слой представления на базе Compose Multiplatform с ViewModel и реактивным управлением состоянием через StateFlow. Реализованы ключевые пользовательские сценарии: регистрация и аутентификация с сохранением сессии, просмотр коллекции паков, отслеживание прогресса с синхронизацией на сервер, добавление паков по уникальному коду, создание и редактирование собственных паков с загрузкой изображений, локализация интерфейса на русский и английский языки. Для отладочных сборок предусмотрена панель переключения API-хоста.

Финальным этапом работы стало тестирование разработанного решения.

Проведены функциональное тестирование основных сценариев использования и кроссплатформенные проверки на целевых платформах (Android, Web и iOS), подтвердившие корректность работы общей логики и пользовательского интерфейса.

Таким образом, все поставленные в работе задачи выполнены. Разработанное приложение Achi удовлетворяет сформулированным требованиям и может быть использовано для отслеживания персональных достижений с элементами геймификации. Перспективными направлениями дальнейшего развития являются реализация полноценного офлайн-режима с локальным хранилищем данных, расширение механизмов обмена паками и добавление in-app переключения языка интерфейса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Achievement Box. Приложение в Google Play. — 2025. — URL: https://play.google.com/store/apps/details?id=hcody.hesham.achivementBox.achivement_box (visited on 06/01/2025).
2. Any.do. Сайт. — 2025. — URL: <https://www.any.do/> (visited on 06/01/2025).
3. Habitica. Сайт. — 2025. — URL: <https://habitica.com/> (visited on 06/01/2025).
4. Notion. Сайт. — 2025. — URL: <https://www.notion.com/> (visited on 06/01/2025).
5. Strides. Сайт. — 2025. — URL: <https://www.stridesapp.com/> (visited on 06/01/2025).
6. The Achievements. Приложение в Google Play. — 2025. — URL: <https://play.google.com/store/apps/details?id=com.mycompany.theachievements> (visited on 06/01/2025).
7. Todoist. Сайт. — 2025. — URL: <https://todoist.com/> (visited on 06/01/2025).

ПРИЛОЖЕНИЯ

Это pdf-файл приложения

Приложение 2

Название приложения

Текст приложения...

Приложение 3

Название приложения

Текст приложения...